Guida Python per la lezione

Strutture di controllo e gestione dei file

Questa guida raccoglie gli undici esempi delle cartelle **02_strutture_di_controllo** e **03_gestione_file**, con il codice completo e le spiegazioni da usare durante la lezione. Il percorso introduce prima le decisioni e le ripetizioni, poi la lettura e la scrittura di dati su disco.

Prerequisiti Variabili, numeri, booleani, stringhe, liste e tuple. Gli esempi richiedono Python 3 e soltanto la libreria standard; non richiedono input da tastiera.

Come usare la guida

Per ogni esempio, chiedere di prevedere il risultato, eseguire lo script e discutere l’output. Le domande e le risposte in fondo alle pagine aiutano a guidare il confronto. Gli esercizi conclusivi permettono di modificare i programmi già funzionanti.

Indice

Argomento	Pagina
Regole comuni delle strutture di controllo	2
1 Condizioni con if elif else	3
2 Iterazione con for e range	4
3 Ripetizione con while	5
4 Simulare il do while	6
5 break continue ed else del ciclo	7
6 Cicli annidati	8
Percorsi e modalità di apertura dei file	9
Testo byte e formato dei dati	10
7 Scrivere e leggere file di testo	11
8 Righe e aggiunta in coda	12
9 File binari e cursore	13
10 Numeri binari con struct	14
11 Gestire gli errori	15
Esercizi e risultati da verificare	16

Regole comuni delle strutture di controllo

Un programma normalmente esegue le istruzioni dall'alto verso il basso. Le strutture di controllo permettono di scegliere un ramo oppure ripetere un blocco. Il simbolo **:** introduce il blocco; l'indentazione ne delimita le istruzioni. Usare quattro spazi per livello, senza mescolare spazi e tabulazioni.

```
voto = 24
if voto >= 18:
    print("Esame superato")
else:
    print("Esame non superato")
print("Questa istruzione viene eseguita in entrambi i casi")
```

Confronti e operatori logici

= assegna un valore; **==** confronta due valori. Gli altri confronti sono **!=, <, <=, >, >=**. Nelle condizioni degli esempi, **and** richiede entrambe le condizioni, **or** almeno una e **not** nega il risultato. **in** controlla l'appartenenza a una raccolta. Le parentesi rendono più chiara una condizione composta.

Scegliere il costrutto

Costrutto	Quando usarlo	Regola da ricordare
if / elif / else	Scegliere tra casi alternativi	Si esegue al massimo un ramo della catena.
for	Visitare gli elementi di un iterabile	range esclude il limite finale; il passo non può essere zero.
while	Ripetere finché una condizione è vera	Il controllo avviene prima del corpo, che può non essere eseguito.
while True con break	Eseguire almeno una volta e poi decidere	Python non ha una specifica istruzione do-while.

Interrompere o saltare

break termina il ciclo più interno. **continue** salta il resto dell'iterazione corrente. L'**else del ciclo** viene eseguito alla conclusione normale, senza break, anche se il corpo ha eseguito zero iterazioni. **pass** non fa nulla: è un segnaposto per un blocco ancora da scrivere e non equivale a continue.

Controllo in aula Provare valori sul confine di una condizione e raccolte vuote. Nei while verificare sempre che una variabile cambi o che un break possa terminare il ciclo.

1 Condizioni con if elif else

Script 02_strutture_di_controllo/01_if_elif_else.py

La prima selezione distingue maggiorenne e minorenne. La catena sul voto verifica prima la validità, poi le soglie: l’ordine delle condizioni determina il ramo eseguito. Gli if separati sulla prenotazione sono indipendenti. La verifica con in riconosce il fine settimana.

```
"""Lezione 1 — Selezione e condizioni."""

print("=== if / else ===")
eta = 20
if eta >= 18:
    print("Maggiorenne")
else:
    print("Minorenne")

print("\n=== if / elif / else: conta l'ordine ===")
voto = 24
if voto < 0 or voto > 30:
    print("Voto non valido")
elif voto < 18:
    print("Esame non superato")
elif voto < 27:
    print("Esame superato")
else:
    print("Ottimo risultato")

print("\n=== Operatori logici e appartenenza ===")
ha_prenotazione = True
documento_valido = True
if ha_prenotazione and documento_valido:
    print("Accesso consentito")
if not ha_prenotazione:
    print("Occorre prenotare")

giorno = "lunedì"
if giorno in ["sabato", "domenica"]:
    print("Fine settimana")
else:
    print("Giorno feriale")

# Esercizio: prova anche 0 e un numero negativo. Quale ramo cambia?
numero = 7
if numero < 0:
    categoria = "negativo"
elif numero == 0:
    categoria = "zero"
else:
    categoria = "positivo"
print(f"\nEsercizio — {numero} è {categoria}")
```

Risultati da osservare Con voto = 24 si stampa «Esame superato». Con numero = 7 la categoria è «positivo».

Domanda in aula Quale ramo viene eseguito per voto = 18?
Risposta Il ramo voto < 27: la condizione voto < 18 è falsa. Provare anche 17, 27 e 31.

2 Iterazione con for e range

Script 02_strutture_di_controllo/02_for_e_range.py

for visita direttamente i nomi della lista. range(1, 6) produce gli interi da 1 a 5; con passo negativo si ottiene il conto alla rovescia. enumerate fornisce posizione e nome. Nell'esercizio, % calcola il resto della divisione: i pari hanno resto zero e vengono aggiunti all'accumulatore.

```
"""Lezione 2 — Iterare su raccolte e intervalli."""

print("=== for su una lista ===")
studenti = ["Anna", "Luca", "Sara"]
for nome in studenti:
    print(f"Ciao, {nome}!")

print("\n=== range: il limite finale è escluso ===")
for numero in range(1, 6):
    print(f"{numero} al quadrato = {numero ** 2}")

print("\n=== Passo negativo ===")
for secondo in range(3, 0, -1):
    print(secondo)
print("Via!")

print("\n=== enumerate: posizione ed elemento ===")
for posizione, nome in enumerate(studenti, start=1):
    print(f"{posizione}. {nome}")

# Esercizio: estendi il limite a 20 e prevedi il nuovo risultato.
somma = 0
for numero in range(1, 11):
    if numero % 2 == 0:
        somma += numero
print(f"\nEsercizio — somma dei pari da 1 a 10: {somma}")
```

Risultati da osservare Si visitano tre studenti; il conto alla rovescia stampa 3, 2, 1. La somma dei pari da 1 a 10 è 30.

Domanda in aula Perché range(1, 6) non include 6?
Risposta Il limite finale è escluso. Per includere 20 nella somma usare range(1, 21): il risultato diventa 110.

3 Ripetizione con while

Script 02_strutture_di_controllo/03_while.py

Il while controlla la condizione prima di ogni iterazione. Il contatore parte da 1 e viene incrementato nel corpo. Il caso con numero = 0 mostra una condizione subito falsa. Nel terzo esempio, totale è un accumulatore: il ciclo continua fino al raggiungimento della soglia.

```
"""Lezione 3 — Ripetere finché una condizione è vera."""

print("=== Contatore: inizializzazione, condizione, aggiornamento ===")
contatore = 1
while contatore <= 5:
    print(f"Iterazione {contatore}")
    contatore += 1  # Senza aggiornamento la condizione resterebbe vera.
print(f"Alla fine il contatore vale {contatore}")

print("\n=== Un while può eseguire zero iterazioni ===")
numero = 0
while numero > 0:
    print("Questo messaggio non viene stampato")
    numero -= 1
print("La condizione iniziale era falsa")

print("\n=== Accumulatore: raggiungere una soglia ===")
totale = 0
numero = 1
while totale < 15:
    totale += numero
    print(f"Aggiunto {numero}; totale = {totale}")
    numero += 1

# Esercizio: cambia la partenza e stampa anche il valore finale.
secondi = 3
print("\nEsercizio — conto alla rovescia")
while secondi > 0:
    print(secondi)
    secondi -= 1
print("Via!")
```

Risultati da osservare Il primo ciclo stampa cinque iterazioni e termina con contatore = 6. L’accumulatore raggiunge 15 sommando 1, 2, 3, 4 e 5.

Domanda in aula Cosa succede togliendo contatore += 1?
Risposta La condizione rimane vera e il ciclo non termina. Fare prevedere il comportamento senza eseguire quel ciclo infinito.

4 Simulare il do while

Script 02_strutture_di_controllo/04_simulare_do_while.py

Python non ha do-while. while True esegue il corpo, poi un if con break decide l'uscita. Per simulare un do-while si esce quando la condizione di continuazione è falsa. La lista dei tentativi sostituisce l'input: deve contenere un valore positivo prima di esaurirsi, altrimenti si ottiene IndexError.

```
"""Lezione 4 — Python non ha do-while: simulazione con while True."""

print("=== while: controllo prima del corpo ===")
numero = 0
while numero > 0:
    print(f"Valore: {numero}")
    numero -= 1
print("Con valore iniziale 0, il corpo non è stato eseguito")

print("\n=== Simulazione do-while: controllo dopo il corpo ===")
numero = 0
while True:
    print(f"Valore: {numero}")
    numero -= 1
    if numero <= 0:  # Esci quando la continuazione (numero > 0) è falsa.
        break
print("Con valore iniziale 0, il corpo è stato eseguito una volta")

print("\n=== Esercizio — ripetere una richiesta fino a un valore valido ===")
# Valori preparati per rendere lo script eseguibile senza input da tastiera.
# Esercizio: aggiungi altri numeri non positivi prima del valore positivo.
tentativi = [-3, 0, 5]
indice = 0
while True:
    numero = tentativi[indice]
    indice += 1
    print(f"Tentativo {indice}: {numero}")
    if numero > 0:
        break
    print("Valore non valido: riprova")
print(f"Numero accettato: {numero}")
# La sequenza deve contenere un numero positivo, altrimenti si esaurisce
# e l'accesso tentativi[indice] produce IndexError.
```

Risultati da osservare Con numero iniziale 0, il while ordinario non esegue il corpo; la simulazione lo esegue una volta. I tentativi -3 e 0 vengono rifiutati; 5 viene accettato.

Domanda in aula Come rendere interattiva la richiesta?
Risposta Sostituire la lettura della lista con int(input(...)); introdurre ValueError per i testi che non rappresentano un intero.

5 break continue ed else del ciclo

Script 02_strutture_di_controllo/05_break_continue_else.py

La ricerca di 10 termina con break e salta l’else del for. continue scarta le misure negative. Nel while l’incremento avviene prima del possibile continue, così il contatore avanza anche per i pari. La ricerca di Marco termina senza break ed esegue l’else.

```
"""Lezione 5 — Interrompere o saltare un'iterazione; else del ciclo."""

print("=== break: interrompere una ricerca ===")
numeri = [3, 7, 10, 12]
cercato = 10
for numero in numeri:
    print(f"Esamino {numero}")
    if numero == cercato:
        print("Trovato: interrompo la ricerca")
        break
else:
    print("Numero non trovato")  # Eseguito soltanto se non c'è stato break.

print("\n=== continue: ignorare i valori negativi ===")
misure = [12, -1, 18, -5, 20]
totale = 0
for misura in misure:
    if misura < 0:
        continue
    totale += misura
    print(f"Misura accettata: {misura}")
print(f"Totale delle misure valide: {totale}")

print("\n=== continue in un while: aggiornare prima del salto ===")
numero = 0
while numero < 5:
    numero += 1
    if numero % 2 == 0:
        continue
    print(f"Dispari: {numero}")

# Esercizio: cerca anche "Sara" e ripeti con una lista vuota.
print("\nEsercizio — ricerca di un nome")
studenti = ["Anna", "Luca", "Sara"]
cercato = "Marco"
for nome in studenti:
    if nome == cercato:
        print(f"{cercato} è presente")
        break
else:
    print(f"{cercato} non trovato")
```

Risultati da osservare La ricerca non esamina 12. Il totale delle misure valide è 50; il while stampa 1, 3, 5. Marco non viene trovato.

Domanda in aula Quando viene eseguito l’else della ricerca?
Risposta Quando non è stato eseguito break. Avviene anche con una lista vuota; cercando Sara si entra invece nel ramo con break.

6 Cicli annidati

Script 02_strutture_di_controllo/06_cicli_annidati.py

Per ogni riga, il ciclo interno visita tutte le colonne. La griglia ha 2 × 3 = 6 celle. Nel secondo esempio, break interrompe soltanto il ciclo sulle colonne; il ciclo sulle righe prosegue. Nella tabella finale, end evita l'a capo dopo ogni prodotto; print() lo inserisce alla fine della riga.

```
"""Lezione 6 — Un ciclo dentro un altro ciclo."""

print("=== Coordinate di una griglia 2 x 3 ===")
conteggio = 0
for riga in range(1, 3):
    for colonna in range(1, 4):
        print(f"Riga {riga}, colonna {colonna}")
        conteggio += 1
print(f"Celle visitate: {conteggio}")

print("\n=== break termina soltanto il ciclo più interno ===")
for riga in range(1, 3):
    for colonna in range(1, 4):
        if colonna == 2:
            break
        print(f"Riga {riga}, colonna {colonna}")
    print(f"Fine del ciclo interno per la riga {riga}")

# Esercizio: estendi entrambi i range per ottenere una tabella 5 x 5.
print("\nEsercizio — tabella di moltiplicazione 3 x 3")
for primo in range(1, 4):
    for secondo in range(1, 4):
        print(f"{primo * secondo:3}", end=" ")
    print()  # Una nuova riga al termine del ciclo interno.
```

Risultati da osservare La griglia visita sei celle. Il secondo esempio stampa soltanto la colonna 1 per ciascuna riga. La tabella finale contiene nove prodotti.

Domanda in aula Come ottenere una tabella 5 × 5?
Risposta Usare range(1, 6) in entrambi i cicli dell'ultimo esempio: il corpo interno viene eseguito 25 volte.

Percorsi e modalità di apertura dei file

Un file conserva i dati dopo la fine del programma. Il percorso indica dove trovarlo; la modalità stabilisce se leggerlo, crearlo, sovrascriverlo o aggiungere dati. Gli esempi preparano autonomamente i file in **03_gestione_file/dati_generati**, esclusa da Git, e possono essere eseguiti in qualsiasi ordine.

```
from pathlib import Path

cartella = Path(__file__).resolve().parent / "dati_generati"
cartella.mkdir(parents=True, exist_ok=True)
percorso = cartella / "saluto.txt"
```

Path rappresenta il percorso; **/** unisce cartella e nome del file. **__file__** identifica lo script, così i dati si trovano anche avviandolo da una directory diversa. In una console o notebook usare una cartella esplicita, per esempio **Path.cwd()**. Un percorso relativo semplice si riferisce alla directory di lavoro corrente.

Modalità di apertura

Modalità	Comportamento	Se il file non esiste
r	Legge testo; è la modalità predefinita.	FileNotFoundError
w	Scrive testo e cancella subito il contenuto precedente.	Crea il file
a	Aggiunge testo in fondo.	Crea il file
x	Crea un nuovo file; se esiste dà FileExistsError.	Crea il file
rb	Legge byte senza decodifica.	FileNotFoundError
wb	Scrive byte e cancella il contenuto precedente.	Crea il file
ab	Aggiunge byte in fondo.	Crea il file

Le modalità di scrittura non creano le cartelle mancanti. Il suffisso **+**, come in r+, abilita lettura e scrittura insieme; introdurlo dopo aver chiarito il cursore.

Chiusura e dati di esempio

with chiude il file all’uscita dal blocco, anche in caso di eccezione. Dopo la chiusura occorre riaprirlo per leggerlo. Ogni esecuzione ricrea i file del relativo esempio; anche l’esempio sull’append riparte da dati noti. Non conservare dati propri con quei nomi. L’esempio sugli errori usa una directory temporanea che viene rimossa alla fine.

Testo byte e formato dei dati

Leggere e scrivere testo

In modalità testo si usano stringhe **str**. Specificare **encoding="utf-8"** rende esplicita la codifica, anche per le lettere accentate. write scrive il testo ricevuto e writelines scrive una sequenza di stringhe: nessuno dei due aggiunge automaticamente gli a capo.

Operazione	Risultato e comportamento
read()	Legge tutto il contenuto; adatto a piccoli file. Usa memoria proporzionale alla dimensione del file.
readline()	Legge una riga. Restituisce "" alla fine del file; una riga vuota può invece essere "\n".
for riga in file	Legge progressivamente le righe. Gli a capo restano nelle stringhe.
rstrip("\n")	Rimuove gli a capo finali dalla stringa letta, senza eliminare altri spazi.
seek(0)	Riporta la lettura all'inizio. In alternativa, chiudere e riaprire il file.

Ogni lettura avanza il cursore. A fine file una lettura testuale restituisce **""**; una lettura binaria restituisce **b""**.

Dai caratteri ai byte

Un file di testo è memorizzato come byte: la codifica stabilisce come interpretarli come caratteri. **encode** trasforma testo in byte; **decode** esegue la conversione inversa, se i byte sono validi per quella codifica. «è» è un carattere ma occupa due byte in UTF-8.

```
testo = "città"
dati = testo.encode("utf-8")
print(len(testo))              # 5 caratteri
print(len(dati))               # 6 byte
print(dati.decode("utf-8"))    # città
```

Il formato binario deve essere concordato

In modalità binaria si usano **bytes**, senza encoding. wb non converte automaticamente numeri o liste: file.write(42) non è valido. Il formato **<3i** di struct usa tre interi con segno da quattro byte; **<** indica little-endian, con il byte meno significativo per primo. Gli interi ammessi vanno da -2**31 a 2**31 - 1. Il file non descrive da solo il significato dei suoi dati.

Cursore Nel binario tell restituisce la posizione in byte. Nel testo non assumere che le posizioni corrispondano agli indici dei caratteri: intervengono codifica e gestione degli a capo.

Errori Distinguere file assente, permessi insufficienti e contenuto non valido. Gestire eccezioni specifiche; evitare un except generico che nasconda errori di programmazione.

7 Scrivere e leggere file di testo

Script 03_gestione_file/01_file_di_testo.py

Path costruisce il percorso vicino allo script e mkdir crea la cartella dei dati. Il primo with apre in w e scrive due righe UTF-8; all'uscita il file è chiuso. Il secondo with riapre in r. read consuma il contenuto e la lettura successiva trova già la fine del file.

```
"""Lezione 1 — Percorsi e scrittura/lettura di testo con UTF-8."""

from pathlib import Path

cartella = Path(__file__).resolve().parent / "dati_generati"
cartella.mkdir(parents=True, exist_ok=True)
percorso = cartella / "saluto.txt"
print(f"File di esempio: {percorso}")

print("\n=== Scrivere testo: w crea o sovrascrive ===")
with percorso.open("w", encoding="utf-8") as file:
    file.write("Ciao, università!\n")
    file.write("Questa è la seconda riga.\n")
print(f"Il file è chiuso fuori dal with: {file.closed}")

print("\n=== Leggere tutto il contenuto ===")
with percorso.open("r", encoding="utf-8") as file:
    contenuto = file.read()
    fine_file = file.read()
print(contenuto, end="")
print(f"Una seconda lettura a fine file restituisce: {fine_file!r}")

print("\n=== Caratteri e byte sono misure diverse ===")
testo = "città"
print(f"{testo!r}: {len(testo)} caratteri, {len(testo.encode('utf-8'))} byte UTF-8")

# Esercizio: aggiungi una terza riga prima della chiusura del primo with.
print(f"\nEsercizio — righe scritte: {len(contenuto.splitlines())}")
```

Risultati da osservare file.closed è True. Il contenuto contiene due righe; la seconda read restituisce una stringa vuota. «città» ha 5 caratteri e 6 byte UTF-8.

Domanda in aula Che cosa succede aprendo di nuovo in w?
Risposta Il contenuto viene cancellato all'apertura. Aggiungere una terza write nel primo with e verificare che il conteggio passi a 3.

8 Righe e aggiunta in coda

Script 03_gestione_file/02_righe_e_append.py

Il primo blocco in w prepara Anna e Luca; il blocco in a aggiunge Sara senza cancellarli. Gli a capo devono essere scritti esplicitamente. Il for legge progressivamente le righe; rstrip elimina soltanto l'a capo. Una nuova apertura in r riporta la lettura all'inizio per le due readline.

```
"""Lezione 2 — Scrivere righe, aggiungere in coda e leggere a righe."""

from pathlib import Path

cartella = Path(__file__).resolve().parent / "dati_generati"
cartella.mkdir(parents=True, exist_ok=True)
percorso = cartella / "studenti.txt"
print(f"File di esempio: {percorso}")

print("\n=== Preparare un contenuto iniziale ===")
# Il reset con w rende il risultato identico a ogni esecuzione.
with percorso.open("w", encoding="utf-8") as file:
    file.writelines(["Anna\n", "Luca\n"])  # Gli a capo sono espliciti.

print("\n=== Append: conservare il testo e aggiungere in fondo ===")
with percorso.open("a", encoding="utf-8") as file:
    file.write("Sara\n")

print("\n=== Iterare sulle righe ===")
conteggio = 0
with percorso.open("r", encoding="utf-8") as file:
    for posizione, riga in enumerate(file, start=1):
        nome = riga.rstrip("\n")
        print(f"{posizione}. {nome}")
        conteggio += 1

print("\n=== readline: ogni lettura avanza il cursore ===")
with percorso.open("r", encoding="utf-8") as file:
    print(f"Prima riga: {file.readline()!r}")
    print(f"Seconda riga: {file.readline()!r}")

# Esercizio: aggiungi un altro nome nel blocco in modalità a.
print(f"\nEsercizio — numero di righe: {conteggio}")
```

Risultati da osservare Si leggono Anna, Luca e Sara, per un totale di tre righe. readline conserva l'a capo, reso visibile dalla rappresentazione !r.

Domanda in aula writelines aggiunge automaticamente gli a capo?
Risposta No. Ogni stringa deve contenere il proprio separatore. Lo script riparte da w a ogni esecuzione, quindi non accumula duplicati.

9 File binari e cursore

Script 03_gestione_file/03_file_binari.py

bytes costruisce una sequenza di valori tra 0 e 255. wb e rb scrivono e leggono byte senza encoding. La lettura a blocchi usa il while e termina con b"". tell indica la posizione in byte; seek(0) riporta il cursore all'inizio. L'ultimo blocco può essere più corto della dimensione richiesta.

```
"""Lezione 3 — Scrivere e leggere byte senza una codifica testuale."""

from pathlib import Path

cartella = Path(__file__).resolve().parent / "dati_generati"
cartella.mkdir(parents=True, exist_ok=True)
percorso = cartella / "byte_esempio.bin"
print(f"File di esempio: {percorso}")

print("\n=== bytes: valori da 0 a 255 ===")
dati = bytes([0, 10, 127, 128, 255])
print(f"Rappresentazione: {dati!r}")
print(f"Valori numerici: {list(dati)}")
with percorso.open("wb") as file:
    file.write(dati)  # In binario si scrivono bytes, non str.

print("\n=== Leggere tutti i byte ===")
with percorso.open("rb") as file:
    riletti = file.read()
print(f"Dati riletti: {riletti!r}")
print(f"Uguali all'originale: {riletti == dati}")

print("\n=== Leggere a blocchi e spostare il cursore ===")
with percorso.open("rb") as file:
    while True:
        blocco = file.read(2)  # L'ultimo blocco può contenere meno di 2 byte.
        if blocco == b"":
            break
        print(f"Blocco: {list(blocco)}; posizione: {file.tell()}")
    file.seek(0)
    print(f"Primo byte dopo seek(0): {file.read(1)!r}")

# Esercizio: cambia i valori iniziali mantenendoli nell'intervallo 0..255.
print(f"\nEsercizio — dimensione del file: {percorso.stat().st_size} byte")
```

Risultati da osservare I byte riletti sono uguali all'originale. I blocchi sono [0, 10], [127, 128] e [255]; le posizioni sono 2, 4 e 5. Il file occupa 5 byte.

Domanda in aula Qual è la differenza tra str e bytes?
Risposta str rappresenta testo; bytes rappresenta byte. Una stringa deve essere codificata prima di scriverla in modalità binaria.

10 Numeri binari con struct

Script 03_gestione_file/04_numeri_binari_struct.py

Il formato <3i stabilisce tre interi con segno da quattro byte, in ordine little-endian. *temperature passa i tre elementi come argomenti separati a pack. Il lettore usa lo stesso formato; unpack restituisce una tupla. calcsize consente di controllare che il contenuto abbia la dimensione attesa.

```
"""Lezione 4 — Un formato binario esplicito per tre interi."""

from pathlib import Path
import struct

cartella = Path(__file__).resolve().parent / "dati_generati"
cartella.mkdir(parents=True, exist_ok=True)
percorso = cartella / "temperature.bin"
print(f"File di esempio: {percorso}")

print("\n=== pack: dai numeri ai byte ===")
temperature = [-5, 0, 23]
formato = "<3i"  # Little-endian, 3 interi con segno da 4 byte ciascuno.
# *temperature passa i tre elementi come tre argomenti a pack.
dati = struct.pack(formato, *temperature)
print(f"Temperature originali: {temperature}")
print(f"Dimensione prevista: {struct.calcsize(formato)} byte")
print(f"Byte prodotti: {dati.hex(' ')}")
with percorso.open("wb") as file:
    file.write(dati)

print("\n=== unpack: dai byte ai numeri ===")
with percorso.open("rb") as file:
    riletti = file.read()

# unpack richiede esattamente il numero di byte previsto dal formato.
if len(riletti) == struct.calcsize(formato):
    recuperate = struct.unpack(formato, riletti)
    print(f"Temperature recuperate (tupla): {recuperate}")
    print(f"Valori conservati: {list(recuperate) == temperature}")
else:
    print("Dimensione non valida per il formato concordato")

# Esercizio: cambia i tre interi. La dimensione del file cambia?
print(f"\nEsercizio — dimensione effettiva: {percorso.stat().st_size} byte")
```

Risultati da osservare Il file occupa 12 byte e la lettura restituisce (-5, 0, 23). Cambiare i valori, restando nel campo degli interi ammessi, non cambia la dimensione.

Domanda in aula Come sa il lettore quanti byte usare per ogni numero?
Risposta Il formato è concordato nel codice. Il file non contiene una descrizione automatica del significato dei suoi byte.

11 Gestire gli errori

Script 03_gestione_file/05_gestire_errori.py

try racchiude l’operazione che può fallire; except gestisce un tipo di errore preciso. I tre casi distinguono un file assente, un testo non convertibile in intero e una creazione in x su un file esistente. TemporaryDirectory isola i dati di prova e li rimuove all’uscita dal with esterno.

```
"""Lezione 5 — Distinguere errori di accesso ed errori nei dati."""

from pathlib import Path
from tempfile import TemporaryDirectory

cartella = Path(__file__).resolve().parent / "dati_generati"
cartella.mkdir(parents=True, exist_ok=True)

# Una directory temporanea garantisce che il file assente non esista
# e isola i casi di errore; viene rimossa all'uscita dal with esterno.
with TemporaryDirectory(prefix="errori_", dir=cartella) as nome_cartella:
    cartella_temporanea = Path(nome_cartella)
    print(f"Directory temporanea di esempio: {cartella_temporanea}")

    print("\n=== File assente ===")
    assente = cartella_temporanea / "assente.txt"
    try:
        with assente.open("r", encoding="utf-8") as file:
            print(file.read())
    except FileNotFoundError:
        print("Il file richiesto non esiste")
    except PermissionError:
        print("Mancano i permessi per leggere il file")

    print("\n=== File presente, contenuto non valido ===")
    percorso = cartella_temporanea / "numero.txt"
    with percorso.open("w", encoding="utf-8") as file:
        file.write("dodici\n")
    with percorso.open("r", encoding="utf-8") as file:
        testo = file.read().strip()
    try:
        numero = int(testo)
        print(f"Numero letto: {numero}")
    except ValueError:
        print(f"{testo!r} non rappresenta un numero intero")

    print("\n=== x impedisce di sovrascrivere un file esistente ===")
    try:
        with percorso.open("x", encoding="utf-8") as file:
            file.write("42\n")
    except FileExistsError:
        print("Il file esiste già: creazione rifiutata")
    with percorso.open("r", encoding="utf-8") as file:
        print(f"Contenuto conservato: {file.read().strip()!r}")

# Esercizio: sostituisci "dodici" con "12" e prevedi l'output.
print("\nLa directory temporanea è stata rimossa automaticamente")
```

Risultati da osservare Si gestiscono FileNotFoundError, ValueError e FileExistsError. La creazione rifiutata lascia intatto «dodici»; la directory temporanea viene rimossa.

Domanda in aula Un file leggibile contiene sempre dati validi?
Risposta No: «dodici» è testo leggibile ma non può essere convertito con int. Sostituirlo con «12» e prevedere quale messaggio cambia.

Esercizi e risultati da verificare

Prima di modificare il codice, prevedere il risultato. Dopo l’esecuzione confrontare l’output con la previsione e provare almeno un caso limite. I commenti negli script contengono già varianti semplici da proporre durante la spiegazione.

Strutture di controllo

1. Classificare un numero come negativo, zero o positivo.
Verifica Provare -3, 0 e 7; ogni valore deve attivare un solo ramo.

2. Sommare i numeri pari da 1 a 20 con for, % e un accumulatore.
Verifica Usare range(1, 21); la somma attesa è 110.

3. Riscrivere il conto alla rovescia con while.
Verifica Con partenza 3 si stampano 3, 2, 1 e poi «Via!». Provare anche 0.

4. Aggiungere tentativi non validi prima del numero positivo.
Verifica Il valore positivo termina il ciclo. La sequenza deve contenerne almeno uno.

5. Cercare un nome in una lista e usare l’else del ciclo.
Verifica Provare un nome presente, uno assente e una lista vuota.

6. Stampare la tabella di moltiplicazione da 1 a 5.
Verifica I due cicli devono produrre 25 prodotti; l’ultimo è 25.

Gestione dei file

7. Scrivere tre nomi, uno per riga, e rileggerli numerandoli.
Verifica Devono comparire tre righe con numerazione da 1 a 3.

8. Contare le righe con un ciclo senza usare read().
Verifica Un file vuoto ha zero righe; una riga vuota nel file va contata.

9. Aggiungere una riga con a e confrontare con w su un file di prova.
Verifica a conserva le righe precedenti; w cancella il contenuto all’apertura.

10. Confrontare la lunghezza di «città» e dei suoi byte UTF-8.
Verifica Sono rispettivamente 5 caratteri e 6 byte.

11. Copiare un file binario leggendo blocchi di due byte.
Verifica Originale e copia devono contenere esattamente gli stessi byte.

12. Cambiare i tre interi dell’esempio struct.
Verifica Con lo stesso formato <3i, il file resta di 12 byte.

13. Gestire UnicodeDecodeError decodificando b"\xff" come UTF-8.
Verifica Quel byte isolato non rappresenta una sequenza UTF-8 valida.